Indian Institute of Information Technology Allahabad
Department of Information Technology
Cyber Security (IT-30001) | Autumn Semester 2026

Assignment: Web Server Security

Exploitation and Mitigation of Four Vulnerability Categories

Mode	Group assignment (maximum four members per group)
Duration	One week
Submission deadline	**15th November 2026 (Sunday),, by midnight (23:59)**
Instructor	Dr. Soumyadev Maity

Overview

Your group will demonstrate, and then defend against, four categories of web server vulnerabilities covered in class: **SQL injection, OS command injection, Cross-Site Scripting (XSS), and Cross-Site Request Forgery (CSRF).** Each member takes one of these categories and implements one variant of it, exploits it with evidence, and then secures it. Between you, your group covers as many of the four categories as its size allows. The task is a hands-on complement to the lecture material on reconnaissance, vulnerability assessment, and web server security.

Important instructions (read first)

1. **Scope.** Each member implements one variant, chosen from the two options listed for a category in Table 1. A full group of four covers all four categories, one variant each; smaller groups cover as many categories as their size allows (see instruction 3). More than four members - two members may do the same category but mandatorily different variants.
2. **Individual ownership.** Each member must personally implement **one variant, exclusive to the others,** so no two members of the group implement the same variant of the same problem category. To maximise coverage, each member should take a **different category,** so that the group **covers as many of the four vulnerability categories as possible** (a group of four or more covers all four categories, one variant each). The ownership declaration, the video and the viva must make clear which member built which variant.
3. **Mitigation (relaxation).** For the vulnerability you implement, you are required to actually **implement only one mitigation of your choice.** However, you must also **study and explain the basic idea (theory only) of the other mitigation alternatives** discussed in the lecture slides for that vulnerability. See item (e) under Table 2.
4. **Application.** You may place all variants inside a single web application, or use separate small applications, whichever you prefer.
5. **Evaluation.** There will be a viva. Each member must be able to demonstrate their own variant live and answer questions on it individually. Originality of the submission will be verified at the viva.
6. **Authorised testing.** Test only your own group's application. Do not test shared classroom infrastructure or the hosting provider.

Table 1. Variant options (each member implements one)

Vulnerability	Variant 1	Variant 2
SQL injection	Authentication bypass	UNION-based data extraction
OS command injection	In-band (command output returned)	Blind (time-based or out-of-band)
Cross-Site Scripting (XSS)	Stored XSS	Reflected XSS
Cross-Site Request Forgery (CSRF)	GET-based forged action	POST-based forged action (auto-submitting form)

Table 2. What each variant must include (to show in the video and at the viva)

For the variant you implement, be ready to show and explain the following in your video demonstration and at the viva:

(a) Vulnerable implementation. The code path that contains the flaw, clearly identified.

(b) Working exploit. The exact payload or request used, shown running in the video demonstration and defended at the viva.

(c) Root cause. One or two lines on why it works, for example user input concatenated into a query or a command, output rendered without encoding, or a state-changing request accepted without proof of intent.

(d) Mitigation implemented. One fix of your choice, with the same payload now shown to fail (a screenshot of the blocked attempt).

(e) Other mitigations (theory). A short paragraph summarising the other mitigation options from the slides for that vulnerability, and why they work. For reference:

SQL injection: parameterised queries, input validation with allow-lists, least-privilege database accounts, safe errors.

Command injection: avoid the shell, parameterised execution with an argument vector, allow-list validation, least privilege.

XSS: context-aware output encoding, framework auto-escaping, a sanitiser for rich HTML, Content Security Policy, HttpOnly cookies, safe DOM sinks.

CSRF: anti-CSRF (synchronizer) tokens, SameSite cookies, re-authentication for sensitive actions, avoiding state-changing GET requests.

Deliverables

1. **Source code** of the application(s) in a public GitHub repository your group creates. Share the repository link as instructed on the course page.
2. **A group video demonstration** (a screen recording with voice-over) in which each member demonstrates their own variant: the working exploit, and the one implemented mitigation blocking the same payload. Each member must narrate their own portion so that individual contributions are clear. Allow roughly two to three minutes per member. Submit the video, or an unlisted link to it, as instructed on the course page.
3. **An ownership declaration** (Table 3), included in the repository, for example in the README, mapping each variant to the member who implemented it.
4. **Readiness to demonstrate and answer** on every variant at the viva.

Table 3. Ownership (to be completed by the group)

Member name	Roll number	Category	Variant implemented

Marks and viva

The viva is individual: each member's score depends on demonstrating and explaining their own variant, including the one implemented mitigation and the theory of the alternatives.

Academic integrity. You may discuss concepts, but the code and exploits submitted must be your group's own work. Any member unable to explain their own variant at the viva will be marked accordingly.